Java Inheritance – Complete Easy Notes

Core Java | Simple English | Beginner Friendly | Exam Preparation

1. Types of Inheritance

Inheritance is a feature of Java in which one class acquires the properties and methods of another class.

- Parent class: Superclass
- Child class: Subclass
- Keyword: extends

There are three main types of inheritance supported through Java classes.

A. Single Inheritance

When one child class inherits one parent class, it is called single inheritance.

Class Diagrams in OOP: Visualizing Your Python Classes - Euron Daily | Euron Community

Example:

```
class Animal {
  void eat() {
    System.out.println("Animal eats");
  }
}

class Dog extends Animal {
  void bark() {
    System.out.println("Dog barks");
  }
}

class Main {
```

```
  public static void main(String[] args) {
    Dog d = new Dog();
    d.eat();
    d.bark();
  }
}
```

Output:

Animal eats

Dog barks

B. Multilevel Inheritance

When a class inherits another child class, it is called multilevel inheritance.

Java Inheritance | PPTX

Example:

```
class Animal {
  void eat() {
    System.out.println("Eating");
  }
}

class Dog extends Animal {
  void bark() {
    System.out.println("Barking");
  }
}

class Puppy extends Dog {
```

```
  void weep() {
    System.out.println("Weeping");
  }
}

class Main {
  public static void main(String[] args) {
    Puppy p = new Puppy();
    p.eat();
    p.bark();
    p.weep();
  }
}
```

Output:

Eating

Barking

Weeping

C. Hierarchical Inheritance

When multiple child classes inherit from one parent class, it is called hierarchical inheritance.

Telusko Docs

Example:

```
class Animal {
  void eat() {
    System.out.println("Eating");
  }
```

```
}

class Dog extends Animal {
  void bark() {
    System.out.println("Barking");
  }
}

class Cat extends Animal {
  void meow() {
    System.out.println("Meowing");
  }
}

class Main {
  public static void main(String[] args) {
    Dog d = new Dog();
    Cat c = new Cat();

    d.eat();
    d.bark();

    c.eat();
    c.meow();
  }
}
```

Output:

Eating

Barking

Eating

Meowing

2. Why Java Classes Do Not Support Multiple Inheritance

Multiple inheritance means one child class inherits from two or more parent classes.

Java does not support multiple inheritance through classes because it can create ambiguity.

Example:

```
class A {
  void show() {
    System.out.println("Class A");
  }
}

class B {
  void show() {
    System.out.println("Class B");
  }
}

// Not allowed in Java
// class C extends A, B { }
```

Problem: If class C calls show(), Java cannot decide whether to execute the method of class A or class B.

This problem is called the Diamond Problem.

Telusko Docs

Solution: Java supports multiple inheritance through interfaces.

```
interface A {
   void show();
}

interface B {
   void display();
}

class C implements A, B {
   public void show() {
      System.out.println("A");
   }

   public void display() {
      System.out.println("B");
   }
}
```

3. IS-A and HAS-A Relationships

These relationships describe how classes are connected.

A. IS-A Relationship

IS-A represents inheritance.

It means one class is a type of another class.

Example: Dog IS-A Animal.

```
class Animal {
```

```
  void eat() {
    System.out.println("Eating");
  }
}

class Dog extends Animal {
}
```

Here, Dog IS-A Animal.

B. HAS-A Relationship

HAS-A represents an association between two classes.

It means one class contains or uses an object of another class.

Example: Car HAS-A Engine.

```
class Engine {
  void start() {
    System.out.println("Engine started");
  }
}

class Car {
  Engine e = new Engine();

  void startCar() {
    e.start();
  }
}
```

Difference:

IS-A	HAS-A
Uses inheritance	Uses object reference
extends keyword	Object creation/reference
Dog IS-A Animal	Car HAS-A Engine

4. Composition in Java

Composition is a strong HAS-A relationship.

In composition, one class contains an object of another class. The contained object is treated as an essential part of the owner object.

Example: House HAS-A Room.

```
class Room {
   void show() {
      System.out.println("Room exists");
   }
}


class House {
   private Room room = new Room();


   void display() {
      room.show();
   }
}


class Main {
   public static void main(String[] args) {
      House h = new House();
```

```
    h.display();
  }
}
```

Output:

Room exists

Advantages:

- Code reusability.
- Better class organization.
- Easy maintenance.
- Reduces dependency on inheritance.

5. super and super() in Java

The super keyword refers to the immediate parent class object.

It is mainly used in three ways.

1. To access parent class variable

```
class A {
  int x = 10;
}


class B extends A {
  int x = 20;


  void show() {
    System.out.println(super.x);
  }
}
```

Output: 10

2. To call parent class method

```
class A {
   void show() {
      System.out.println("Parent method");
   }
}

class B extends A {
   void show() {
      super.show();
      System.out.println("Child method");
   }
}
```

3. To call parent class constructor using super()

```
class A {
   A() {
      System.out.println("Parent constructor");
   }
}

class B extends A {
   B() {
      super();
      System.out.println("Child constructor");
   }
}
```

Output:

Parent constructor

Child constructor

Remember:

- super accesses parent class members.
- super() calls the parent class constructor.
- super() must be the first statement in a constructor when written explicitly.

6. Method Overriding Rules

Method overriding occurs when a child class provides its own implementation of a method already defined in the parent class.

Example:

```
class Animal {
  void sound() {
    System.out.println("Animal sound");
  }
}


class Dog extends Animal {
  @Override
  void sound() {
    System.out.println("Dog barks");
  }
}
```

Output:

Dog barks

Rules of Method Overriding:

1. Method name must be the same.
2. Parameters must be the same.
3. Inheritance is required.
4. Return type must be the same or covariant.
5. Access level cannot be more restrictive.
6. Static methods cannot be overridden.
7. Final methods cannot be overridden.
8. Private methods cannot be overridden.
9. Constructors cannot be overridden.

7. Method Overloading vs Method Overriding

Method Overloading

When multiple methods have the same name but different parameters in the same class, it is called method overloading.

```
class Calculator {
  int add(int a, int b) {
    return a + b;
  }

  int add(int a, int b, int c) {
    return a + b + c;
  }
}
```

Method Overriding

When a child class changes the implementation of a parent class method, it is called method overriding.

```
class Animal {
  void sound() {
```

```
    System.out.println("Animal sound");
  }
}

class Dog extends Animal {
  @Override
  void sound() {
    System.out.println("Dog sound");
  }
}
```

Difference:

Overloading	**Overriding**
Same class is possible	Requires inheritance
Different parameters	Same parameters
Compile-time polymorphism	Runtime polymorphism
Return type alone cannot differentiate methods	Same or covariant return type
Inheritance not required	Inheritance required

8. Dynamic Method Dispatch and Late Binding

Dynamic method dispatch is a process in which a method call is decided at runtime.

It is used to achieve runtime polymorphism.

Example:

```
class Animal {
  void sound() {
    System.out.println("Animal sound");
  }
```

```
}

class Dog extends Animal {
  @Override
  void sound() {
    System.out.println("Dog barks");
  }
}

class Main {
  public static void main(String[] args) {
    Animal a = new Dog();
    a.sound();
  }
}
```

Output:

Dog barks

Explanation:

- Animal is the reference type.
- Dog is the actual object type.
- Java executes the overridden method of Dog at runtime.

Late Binding: When the method call is connected to its actual method implementation at runtime, it is called late binding.

9. Upcasting, Downcasting and ClassCastException

A. Upcasting

Converting a child class object reference into a parent class reference is called upcasting.

It happens automatically.

```
class Animal {
   void eat() {
      System.out.println("Eating");
   }
}


class Dog extends Animal {
   void bark() {
      System.out.println("Barking");
   }
}


class Main {
   public static void main(String[] args) {
      Animal a = new Dog();
      a.eat();
   }
}
```

B. Downcasting

Converting a parent class reference back into a child class reference is called downcasting.

It requires explicit casting.

```
class Main {
   public static void main(String[] args) {
      Animal a = new Dog();
```

```
    Dog d = (Dog) a;
    d.bark();
  }
}
```

C. ClassCastException

ClassCastException occurs when an object is incorrectly converted into an incompatible class type.

Example:

```
class Animal {
}

class Dog extends Animal {
}

class Cat extends Animal {
}

class Main {
  public static void main(String[] args) {
    Animal a = new Dog();

    Cat c = (Cat) a;
  }
}
```

This produces ClassCastException because the actual object is Dog, not Cat.

Safe Downcasting using instanceof:

```
if (a instanceof Dog) {
   Dog d = (Dog) a;
   System.out.println("Safe casting");
}
```

10. Covariant Returns, Static Hiding and Final Methods

A. Covariant Return Type

When an overridden method returns a subclass type instead of the parent method's return type, it is called a covariant return type.

Example:

```
class Animal {
   Animal getObject() {
      return this;
   }
}


class Dog extends Animal {
   @Override
   Dog getObject() {
      return this;
   }
}
```

Here, Dog is a subclass of Animal, so the return type Dog is valid.

B. Static Method Hiding

When a child class defines a static method with the same signature as a static method in the parent class, it is called static method hiding.

Example:

```
class A {
   static void show() {
      System.out.println("Parent");
   }
}

class B extends A {
   static void show() {
      System.out.println("Child");
   }
}

class Main {
   public static void main(String[] args) {
      A.show();
      B.show();
   }
}
```

Output:

Parent

Child

Remember: Static methods are hidden, not overridden.

C. Final Methods

A method declared with the final keyword cannot be overridden by a child class.

Example:

```
class A {
```

```
    final void show() {
        System.out.println("Final method");
    }
}

class B extends A {
    // Cannot override show()
}
```

Uses of final:

- Prevents method overriding.
- Provides controlled inheritance.
- Helps protect important method implementations.